**University of Pretoria**
Department of Computer Science

# COS 730 – Assignment 1

## Advanced Behavioural Modelling and Responsibility-Driven Design

| | |
|---|---|
| **Module** | COS 730 – Advanced Software Engineering |
| **Assignment** | 1 |
| **Student Name** | Yohali Malaika Kamangu |
| **Student Number** | u23618583 |
| **Date Submitted** | 19 March 2026 |
| **Selected Use Cases** | UC1: Create Research Project<br>UC2: Submit Dataset<br>UC3: Automated Reviewer Assignment |

# Contents

# 1 Introduction

The **Intelligent Research Collaboration Platform (IRCP)** is a complex sociotechnical system designed to support the full lifecycle of academic research activities — from project inception through to dataset management, peer review, collaboration monitoring, and output evaluation. The platform must orchestrate multiple interacting subsystems with clearly defined responsibilities, robust validation, and transparent audit trails.

For this assignment, three use cases were selected for analysis: **UC1: Create Research Project**, **UC2: Submit Dataset**, and **UC3: Automated Reviewer Assignment**. The selection was deliberate — each one sits at a different point on the spectrum of system behaviour. UC1 is event-driven: a single researcher action triggers a cascade of downstream activity across multiple collaborating services. UC2 works more like a pipeline, moving a raw file through sequential validation and storage stages where each stage is a prerequisite for the next. UC3 is driven entirely by rules — the output depends on which business constraints are satisfied at runtime, not on any fixed data transformation.

The report is organised into four tasks. Task 1 builds sequence diagrams for each use case and examines where their interaction patterns overlap — these redundancies point toward a more generalised architecture. Task 2 applies GRASP principles to justify responsibility assignments, then steps back to evaluate coupling, cohesion, and scalability across the whole system. Task 3 classifies each use case by its behaviour archetype and argues for complementary modelling techniques beyond sequence diagrams. Task 4 synthesises all of this into a design optimisation analysis, with supporting class diagrams for each use case.

# 2 Task 1 – Interaction Modelling

## 2.1 Use Case 1: Create Research Project

### *2.1.1 Formal Use Case Description*

| Field | Description |
|---|---|
| **Use Case Name** | Create Research Project |
| **Use Case ID** | UC1 |
| **Actor(s)** | Researcher (Primary); System (Secondary) |
| **Preconditions** | The researcher is authenticated and has permission to create new projects. All collaborator identifiers provided must correspond to registered researchers in the system. |
| **Postconditions** | A new research project record exists in the persistent store with a unique identifier. Project roles have been assigned to all valid collaborators. All collaborators have been notified. An immutable audit log entry has been created. |

**Main Success Scenario:**

1. The researcher initiates project creation by submitting a `ProjectDetails` object containing the project title, a list of objectives, a set of milestones with target dates, and a list of collaborator identifiers.
2. The system validates the submitted details: mandatory fields (title, at least one

objective) are present; milestone target dates are chronologically consistent and set in the future; all collaborator identifiers resolve to registered researchers.
3. Upon successful validation, the system initialises a new `Project` entity with a generated `projectId` and a status of `DRAFT`.
4. The system persists the `Project` entity to the project repository and receives the persisted entity back with its assigned identifier.
5. The system triggers the `RoleAssigner` to determine and assign appropriate roles (e.g., Principal Investigator for the creating researcher, Co-Investigator or Research Assistant for named collaborators) based on the project context.
6. The system triggers the `NotificationService` to send notifications to all assigned collaborators informing them of their inclusion in the new project.
7. The system creates an audit log entry recording the creation event, the initiating researcher, and the project identifier.
8. The system returns a `ProjectCreationResponse` to the researcher, encapsulating the persisted project and a summary of assigned roles.

**Alternative Flow 1 – Validation Failure:** At step 2, if any mandatory field is absent, if milestone dates are inconsistent, or if a collaborator identifier is unresolvable, the system immediately returns a `ProjectCreationException` that enumerates all validation errors. No `Project` entity is created, no roles are assigned, no notifications are sent, and no audit entry is made. The researcher must correct the errors and resubmit.

**Alternative Flow 2 – Partial Collaborator Resolution:** At step 2, if the system resolves some but not all collaborator identifiers (e.g., one identifier belongs to a deactivated account), validation does not fail outright. The system proceeds with all valid collaborators and includes a `warnings` list in the `ProjectCreationResponse` indicating which identifiers could not be resolved, prompting the researcher to revisit collaborator membership post-creation.

#### *2.1.2 Sequence Diagram*

The following diagram illustrates the message flow for UC1. The `ProjectController` acts as the system's sole entry point for this use case, delegating to six specialised components. Decision points are modelled using `alt` fragments. Activation bars indicate the duration of each object's processing responsibility.

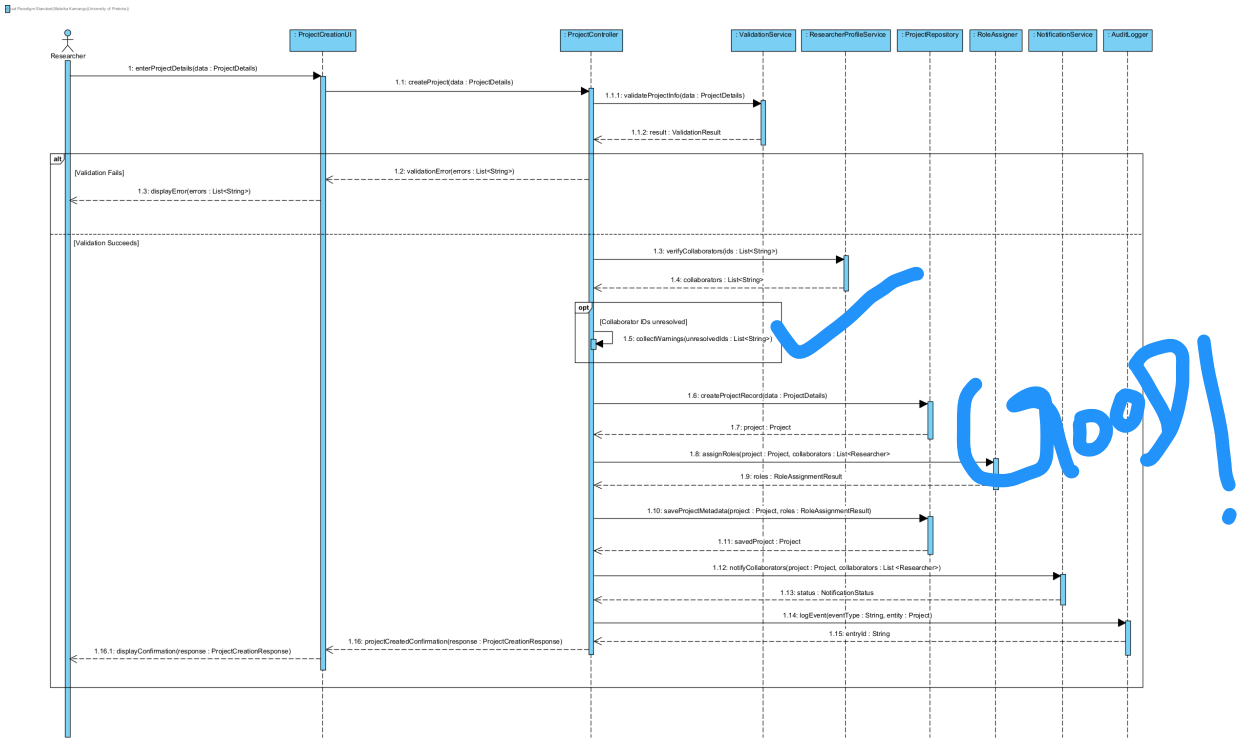


Figure 1: UC1: Create Research Project – Sequence Diagram

**Participating Objects:** `Researcher`, `ProjectCreationUI` (boundary), `ProjectController` (control), `ValidationService`, `ProjectRepository`, `ResearcherProfileService`, `RoleAssigner`, `NotificationService`, `AuditLogger`.

**Message Flows:** The researcher submits project details through `ProjectCreationUI`, which hands the call off to `ProjectController`. Validation is deliberately first: if the input is bad there is nothing further to do, and the `alt` fragment models this cleanly — any failure short-circuits the entire flow and returns a complete error list with no side effects and no partial state written (Alt Flow 1). Assuming the data passes, the controller checks the collaborator IDs against `ResearcherProfileService`. The `opt` fragment captures the nuanced case of partial resolution (Alt Flow 2): rather than rejecting the whole request over a single stale account, the controller collects unresolved IDs as warnings and continues with the valid collaborators. Role assignment, two-stage persistence, notifications, and audit logging then complete the flow, with `ProjectCreationResponse` carrying the persisted project and role summary back to the actor.

**Control Responsibilities:** `ProjectController` is deliberately thin — it sequences the UC1 workflow but implements no domain logic of its own. Validation rules belong to `ValidationService`, role decisions to `RoleAssigner`, notification mechanics to `NotificationService`. The controller knows *what* needs to happen and in what order; it does not know *how* any of it works.

**Decision Points and System Responses:**

- **Decision 1 (`alt`):** Validation is the gatekeeper. A failed `ValidationResult` terminates the flow immediately, returning all errors at once with no partial state created — consistent with Alt Flow 1.

- **Decision 2 (`opt`):** Partial collaborator resolution is treated differently from outright validation failure. If some IDs resolve and some do not, the flow continues with the valid subset and surfaces the unresolved IDs as warnings in the response — avoiding the frustration of blocking project creation over a single deactivated account.
- **Decision 3 (within `RoleAssigner`):** Which role each collaborator receives is entirely `RoleAssigner`'s concern. The controller simply passes the project and collaborator list and accepts the result.

## 2.2 Use Case 2: Submit Dataset

### *2.2.1 Formal Use Case Description*

| Field | Description |
|---|---|
| **Use Case Name** | Submit Dataset |
| **Use Case ID** | UC2 |
| **Actor(s)** | Researcher (Primary); System (Secondary) |
| **Preconditions** | The researcher is authenticated. The referenced research project exists and has `ACTIVE` status. The researcher has dataset submission permissions on the project. |
| **Postconditions** | The dataset is validated, its integrity confirmed, its metadata extracted and stored, and it is linked to the research project. A dataset record with a unique identifier exists in the persistent store. |

**Main Success Scenario:**

1. The researcher initiates dataset submission by providing a data file and the associated project identifier.
2. The system validates the file format against the list of accepted dataset formats (CSV, JSON, HDF5, XML, Parquet).
3. The system performs an integrity scan on the file: it computes a checksum and detects any corruption indicators.
4. The system extracts metadata from the file, including schema structure, record count, file size, and creation timestamp.
5. The system constructs a `Dataset` entity, embedding the extracted metadata and the computed checksum.
6. The system uploads the file to the `StorageService`, which stores the file in persistent blob storage and returns a `storageRef` URI.
7. The system saves the `Dataset` entity — including the `storageRef` — to the dataset repository, receiving back the persisted entity with a unique `datasetId`.
8. The system updates the project record to link the newly created dataset, establishing a persistent association between the dataset and the research project.
9. The system creates an audit log entry recording the submission event, the submitting researcher, and the `datasetId`.
10. The system returns a `DatasetSubmissionResponse` to the researcher confirming the submission and providing the assigned `datasetId`.

**Alternative Flow 1 – Invalid File Format:** At step 2, if the uploaded file's format is not in the accepted list, the system immediately returns a `DatasetSubmissionException("Unsupported`

`file format")`. No integrity scan, metadata extraction, or storage operation is performed. The researcher is advised to convert the file to a supported format and resubmit.

**Alternative Flow 2 – Integrity Check Failure:** At step 3, if the integrity scanner detects file corruption (e.g., truncated records, mismatched checksum, binary inconsistency), the system flags the dataset as `QUARANTINED`, records a quarantine entry with the detected issues, and returns a `DatasetSubmissionException("Integrity check failed")`. No metadata is extracted and no storage occurs. The researcher is advised to re-upload an uncorrupted copy.

### *2.2.2 Sequence Diagram*

The UC2 sequence diagram models a sequential validation pipeline in which the file must pass two guarded stages before any persistent operation is performed. The two `alt` fragments at the head of the flow act as guards: format validation runs first (cheapest), then integrity scanning (expensive). Only after both pass does the pipeline proceed to metadata extraction, two-step storage, project linking, and audit logging.

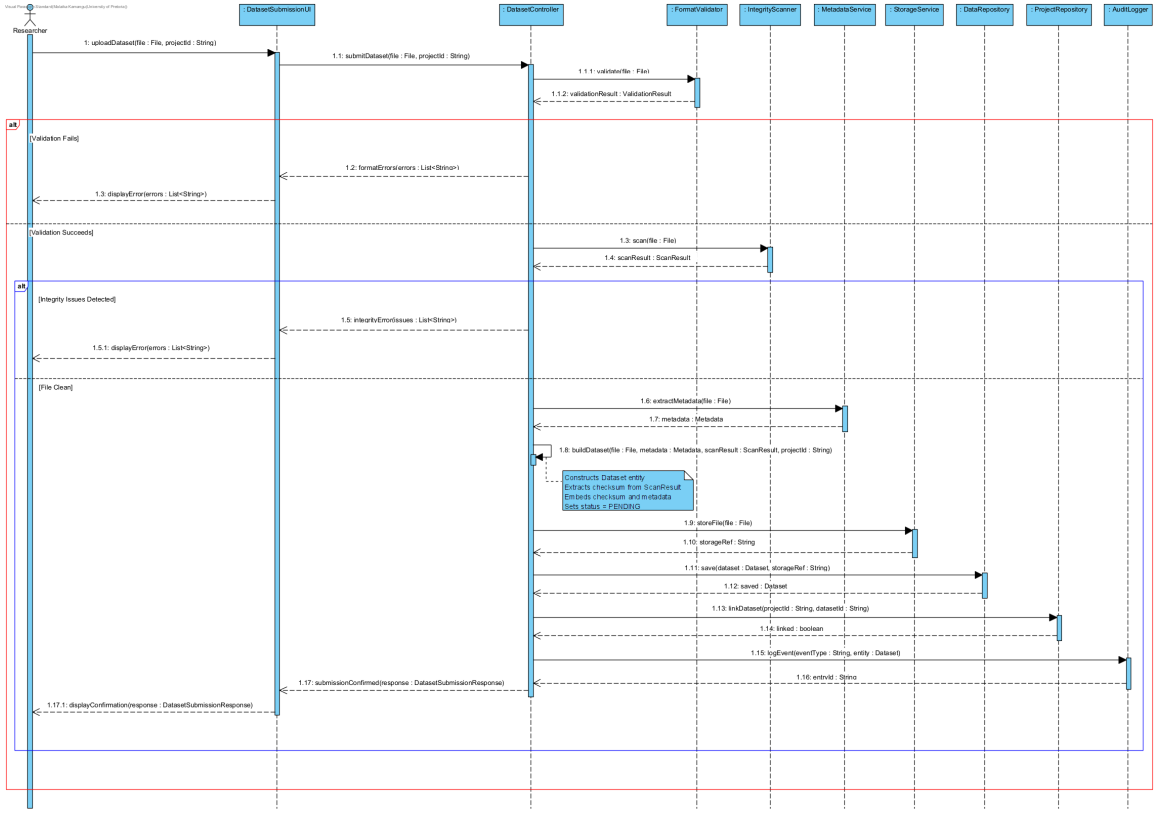


Figure 2: UC2: Submit Dataset – Sequence Diagram

**Participating Objects:** `Researcher`, `DatasetSubmissionUI` (boundary), `DatasetController` (control), `FormatValidator`, `IntegrityScanner`, `MetadataService`, `StorageService`, `DatasetRepository`, `ProjectRepository`, `AuditLogger`.

**Message Flows:** The researcher uploads a file through `DatasetSubmissionUI`, which forwards to `DatasetController`. The controller immediately calls `FormatValidator.validate()` — there is no point scanning a file for integrity issues if its format is not supported. The outer `alt` block branches on this result. Inside the valid-format branch, `IntegrityScanner.scan()` runs next; the inner `alt` block then branches on the scan result. Only in the clean-file

branch does the expensive work begin: `MetadataService` extracts schema and record-count information, the controller assembles the `Dataset` entity in a self-call, `StorageService` writes the raw file and returns a `storageRef`, `DatasetRepository` persists the record with that reference, `ProjectRepository` links the dataset to the project, and `AuditLogger` closes the flow.

**Control Responsibilities:** `DatasetController` is the mediator for the entire UC2 pipeline. It passes data between stages — the checksum from `IntegrityScanner`, the metadata from `MetadataService`, the storage reference from `StorageService` — without any individual stage knowing about the others. Nothing is persisted until both validation gates have passed, a constraint that only the controller can enforce because it is the only participant with visibility across the whole flow.

**Decision Points and System Responses:**

- **Decision 1 (`alt` – Validation Fails / Validation Passes):** Format validation is the cheapest gate and runs first. On failure the whole flow stops — no integrity scan, no storage, no database writes. This is the principle of failing fast applied directly in the sequence structure.
- **Decision 2 (`alt` – Integrity Issues Detected / File Clean):** A corrupted file that happens to be the right format would slip through format validation. The integrity scan catches it here. On failure the flow again terminates cleanly, before any storage I/O occurs.
- **Decision 3 (implicit, within `StorageService`):** Separating blob storage from relational persistence is a key structural decision. `StorageService` handles the raw file and returns a `storageRef` URI; `DatasetRepository` stores only the lightweight metadata record pointing at that reference. This means the two stores can scale, migrate, or fail independently.

## 2.3 Use Case 3: Automated Reviewer Assignment

### *2.3.1 Formal Use Case Description*

| Field | Description |
|---|---|
| **Use Case Name** | Automated Reviewer Assignment |
| **Use Case ID** | UC3 |
| **Actor(s)** | System (Primary — event-triggered); Administrator (Observer) |
| **Preconditions** | A research output has been submitted and its record exists in the system. The reviewer pool contains registered reviewers with defined expertise domains. The system has a configured minimum reviewer count per output. |
| **Postconditions** | A set of reviewers have been assigned to the research output, free of conflicts of interest, with balanced workloads. The assignment is recorded with a rationale string. All assigned reviewers have been notified. An audit log entry records the assignment event. |

**Main Success Scenario:**

1. The system receives an event trigger upon research output submission, containing the output identifier.

2. The `ReviewerAssignmentController` retrieves the `ResearchOutput` details — domain, author list, and title — from the `ResearchOutputRepository`.
3. The `ReviewerPool` identifies candidate reviewers whose declared expertise domains match the output's research domain.
4. The `ConflictChecker` applies conflict-of-interest rules to filter the candidate list, removing reviewers who are co-authors of the output, belong to the same institution as an author, or have had a recent collaboration with an author (within a configurable recency window).
5. The `WorkloadBalancer` selects the required number of reviewers from the conflict-free list, prioritising those with the lowest current assignment workload, provided they are below the maximum workload threshold.
6. The `ReviewerAssignmentController` constructs an `Assignment` entity containing the output identifier, the selected reviewer identifiers, a timestamp, and a rationale string documenting the selection logic applied.
7. The `AssignmentRepository` persists the `Assignment` entity.
8. The `NotificationService` dispatches assignment notifications to each selected reviewer.
9. The system creates an audit log entry recording the assignment event, the `assignmentId`, and the assigned reviewer identifiers.
10. The controller emits an `AssignmentResponse` to the triggering system, confirming the assignment identifier and the reviewer list.

**Alternative Flow 1 – Insufficient Eligible Reviewers:** At step 5, if the conflict-free, available reviewer pool contains fewer than the required minimum count, the system raises an `AssignmentException`. No assignment is persisted. The system creates a manual review queue entry for administrator resolution, and the research output's status is updated to `PENDING_MANUAL_ASSIGNMENT`.

**Alternative Flow 2 – No Domain-Matching Reviewers:** At step 3, if the `ReviewerPool` finds no reviewers with expertise matching the output's specific research domain, the system attempts a domain relaxation strategy: it widens the search to the parent domain category (e.g., widening from "Quantum Computing" to "Computer Science"). If domain relaxation yields candidates, the flow continues with those candidates from step 4. If no candidates are found even after relaxation, Alternative Flow 1 is triggered.

### *2.3.2 Sequence Diagram*

UC3 is the only system-triggered use case among the three selected. The initiating actor is the system itself (an event published upon research output submission), rather than a human researcher. This distinction is reflected in the sequence diagram by using an `AssignmentScheduler` participant rather than an actor.

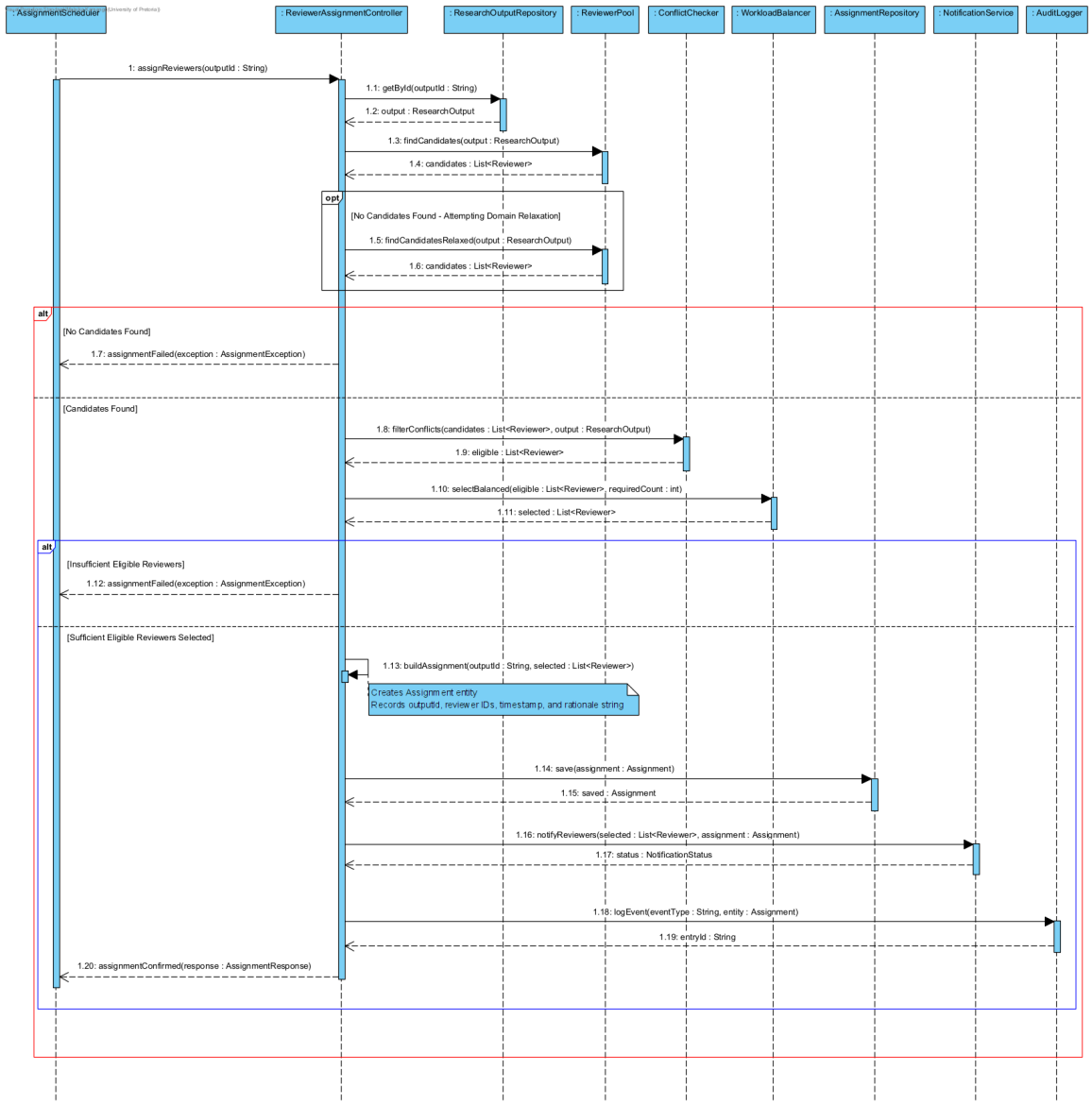


Figure 3: UC3: Automated Reviewer Assignment – Sequence Diagram

**Participating Objects:** `AssignmentScheduler`, `ReviewerAssignmentController`, `ResearchOutputReposi` `ReviewerPool`, `ConflictChecker`, `WorkloadBalancer`, `AssignmentRepository`, `NotificationService`, `AuditLogger`.

**Message Flows:** The trigger passes only the output ID to `ReviewerAssignmentController`, which must first fetch the full output context — domain, authors, title — from `ResearchOutputRepository`. With that context in hand, `ReviewerPool.findCandidates()` searches for domain-matching reviewers. The `opt` fragment immediately after handles the empty-result case: `findCandidatesRelaxed()` widens the search to the parent domain category, giving the system one more chance to find suitable reviewers before giving up (Alt Flow 2). The outer `alt` then branches: an empty candidate list at this point triggers `assignmentFailed` immediately. In the candidates-found branch, `ConflictChecker.filterConflicts()` applies the conflict rules to produce an eligible shortlist, and `WorkloadBalancer.selectBalanced()` makes the final selection. The inner `alt` guards against an insufficient count (Alt Flow 1). When the count is sufficient, a self-call on the controller builds the `Assignment` entity with rationale

attached, before persistence, notifications, and audit logging complete the flow.

**Control Responsibilities:** `ReviewerAssignmentController` orchestrates the workflow but applies no rules itself. The conflict-of-interest logic lives entirely in `ConflictChecker`, workload selection in `WorkloadBalancer`, and candidate discovery in `ReviewerPool`. The controller knows the sequence; the specialist classes know the rules.

**Decision Points and System Responses:**

- `opt` — **Domain Relaxation:** Rather than failing immediately when no domain-matching reviewers exist, the system makes one further attempt by widening the search to the parent domain category. The `opt` fragment keeps this as a conditional step — it only runs when the initial search returns nothing, and it updates the `candidates` list in place for the outer `alt` to evaluate.
- **Decision 1 (`alt` — No Candidates / Candidates Found):** Once the optional relaxation pass is complete, the outer `alt` evaluates the final candidate list. An empty list at this point means assignment cannot proceed: `assignmentFailed` is returned and nothing is written to the database.
- **Decision 2 (`alt` — Insufficient Eligible Reviewers / Sufficient):** Having candidates is not enough — they must also pass conflict filtering and meet the minimum count after workload selection. The inner `alt` enforces this. Falling below the configured threshold triggers `assignmentFailed` without persisting an incomplete assignment.

### 2.4 Modelling Redundancies and Refined Interaction Approach

Looking across all three sequence diagrams, four structural patterns repeat themselves in ways that suggest opportunities for a more generalised design. These are not accidental similarities — they reflect the fact that all three use cases must validate input, persist an entity, and record an audit trail.

**Redundancy 1: The Validation Pattern.** Both UC1 and UC2 follow the same three-step interaction for validation: delegate to a specialist validator, receive a `ValidationResult`, terminate the flow if it signals failure. The participating classes differ (`ValidationService` vs. `FormatValidator`) and the input types differ (`ProjectDetails` vs. `File`), but the interaction fragment is otherwise identical.

**Redundancy 2: The Repository Save Pattern.** Persistence follows the same structural beat in all three use cases: pass a domain entity to a repository, get back the persisted version with an assigned identifier. UC1 does this in two steps (initial record creation, then metadata attachment), while UC2 and UC3 do it in one. The underlying interaction contract is the same; a shared `Repository<T>` interface would make that explicit.

**Redundancy 3: NotificationService Reuse.** `NotificationService` serves both UC1 and UC3 with what is effectively the same interaction: the controller provides a target list and an event, the service delivers, and returns a `NotificationStatus`. The method names differ but the pattern is identical. That UC2 has no notification step at all makes the repetition across the other two even clearer.

**Redundancy 4: Audit Logging Pattern.** Every use case ends with the same audit call: `AuditLogger.logEvent(eventType, entity)` returns an `entryId` and the controller proceeds to return its response. This is the simplest of the four patterns to unify — it is

literally the same interaction appearing three times with different values plugged in.

**Proposed Refined Interaction Approach:**

Taken together, these redundancies point toward an abstract `BaseController` as the anchor of a more generalised architecture. The refined model introduces three abstractions. A `Validator<T>` interface unifies all validation classes under a single contract, so the validation interaction pattern can be described once and referenced in each use case's diagram via `«ref»`. A `Repository<T>` interface normalises the persistence contract across all three repository classes. The abstract `BaseController` injects `NotificationService` and `AuditLogger` as inherited fields, removing the need to wire them into each concrete controller separately:

1. A `Validator<T>` interface unifies `ValidationService` and `FormatValidator` under one contract, allowing the validation pattern to be described once and referenced elsewhere.
2. A `Repository<T>` interface normalises the save/find/update contract across `ProjectRepository`, `DatasetRepository`, and `AssignmentRepository`.
3. A `BaseController` inherits `NotificationService` and `AuditLogger` as shared fields, so concrete controllers do not independently wire these cross-cutting services.
4. `«ref»` sequence fragments in diagrams reference the shared patterns described in a canonical interaction overview, eliminating repetition at the modelling level too.

In practice, a change to the audit entry format propagates automatically to every use case because the call is defined once in `BaseController`. The same holds for notification dispatch. New use cases inherit these behaviours without additional effort.

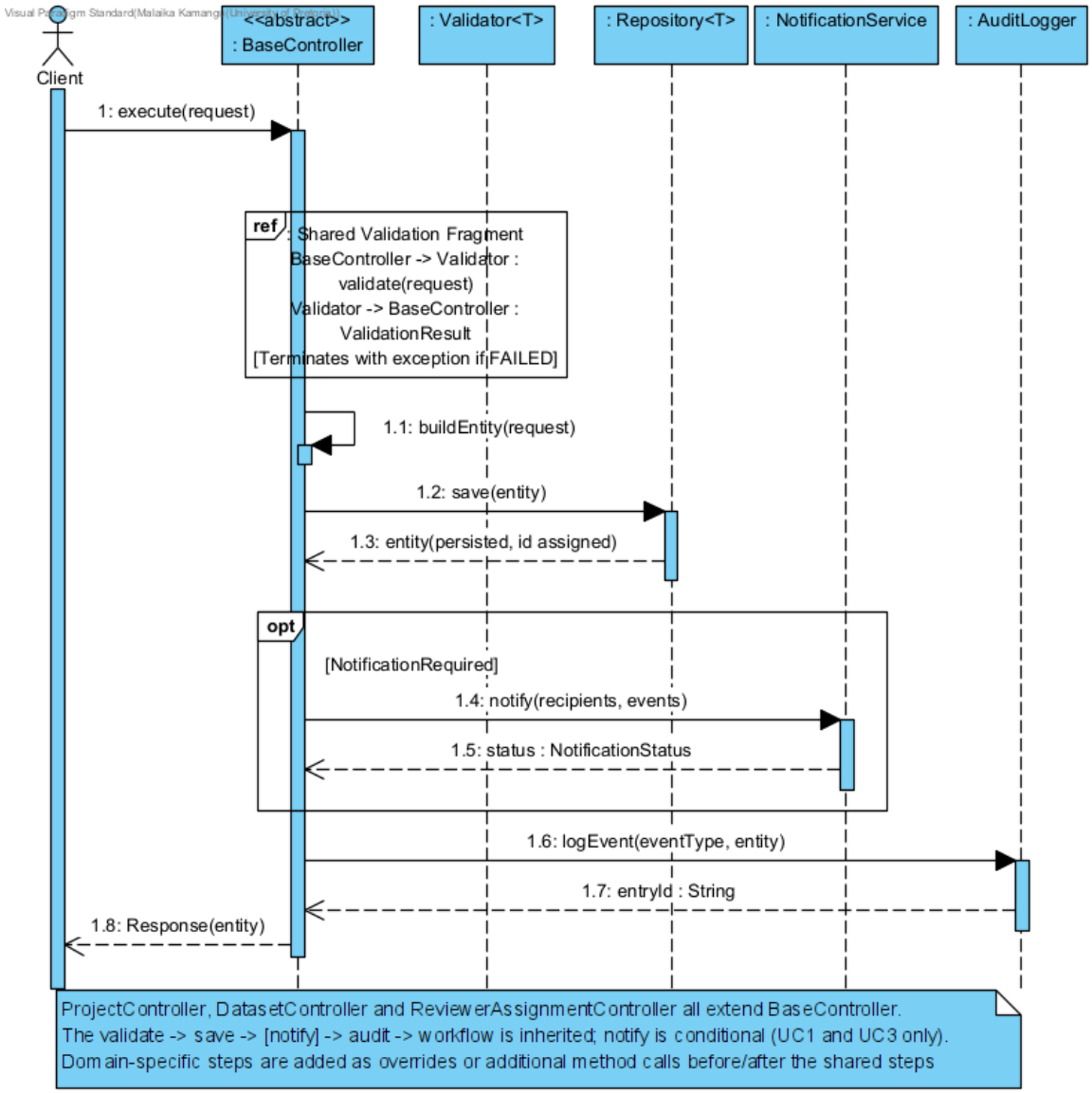


Figure 4: Refined Interaction Model: Shared `BaseController` Workflow

# 3 Task 2 – Responsibility Assignment Analysis

This section assigns responsibilities to system components using GRASP principles and evaluates the design choices for each use case. A system-wide evaluation of coupling, cohesion, and scalability follows the per-use-case analyses.

## 3.1 UC1: Create Research Project

**Controller.** `ProjectController` takes the Controller role for UC1. The GRASP principle here is straightforward: the system event (`createProject()`) should be received by a dedicated use-case controller, not scattered across domain objects. Without this clear entry point, the `Researcher` actor would need direct knowledge of `ValidationService`, `ProjectRepository`, and every other collaborator — an unworkable coupling that the controller exists to prevent.

**Creator.** The Creator designation also falls on `ProjectController`. It receives the `ProjectDetails` from which the `Project` is constructed, it starts the entity's lifecycle, and

it hands the created object to downstream services — all three of the conditions GRASP uses to identify a natural creator. Keeping creation here means no collaborating service ever needs to know how a `Project` gets built.

**Information Expert.** Three Information Expert assignments are worth highlighting in UC1. `ValidationService` owns the knowledge of what makes a project valid — mandatory fields, date consistency, collaborator resolution logic. `ProjectRepository` knows how to persist and retrieve project data; the controller has no view into how that works. `RoleAssigner` holds the mapping between researcher attributes and project roles, keeping that business logic isolated from the workflow coordinator.

**Indirection.** Indirection is applied through `NotificationService`, which sits between `ProjectController` and whatever delivery mechanism actually sends notifications. The controller calls a single method without knowing whether the result is an email, a push notification, or both. Changing or extending the delivery mechanism is a matter of modifying `NotificationService` only.

**Polymorphism.** Polymorphism enters through the `Validator<ProjectDetails>` interface that `ValidationService` implements. This means a different validator — say, a stricter `EthicsApprovalValidator` for funded research — can be injected at runtime without touching the controller. The controller simply invokes the contract; what happens behind it can vary.

### 3.2 UC2: Submit Dataset

**Controller.** `DatasetController` handles the Controller role for UC2. Its job is sequencing: call each pipeline stage in order, collect the results, and pass them onward. It has no opinion on how formats are detected, how checksums are computed, or how files are stored — those concerns belong to the specialist classes.

**Creator.** `DatasetController` creates the `Dataset` entity because it is the only class that holds all of the necessary inputs at the same time — the file reference, the extracted metadata, the integrity checksum, and the project identifier. The private `buildDataset()` method formalises this assembly step, keeping construction logic in one place.

**Information Expert.** UC2 distributes information expertise across six classes. `FormatValidator` knows which formats are accepted. `IntegrityScanner` carries the checksum algorithms and corruption heuristics. `MetadataService` understands how to parse schema structure and count records across different file types. `StorageService` handles the blob storage layer and generates storage references. `DatasetRepository` manages persistence of the dataset record itself, and `ProjectRepository` owns the project–dataset association logic.

**Indirection.** Indirection is pervasive in UC2. `StorageService` hides the file storage infrastructure (local disk, cloud blob, network share — the controller does not care which). `MetadataService` hides the file-parsing libraries or format-specific APIs. `DatasetRepository` abstracts the database layer. Because none of these expose implementation details upward, replacing any one of them is a contained change.

**Polymorphism.** Two polymorphism points stand out in UC2. `FormatValidator` implements `Validator<File>`, which means a stricter or more lenient format policy is simply a matter of swapping implementations. `StorageService` implements a `FileStorage` interface, so migrating from a local filesystem to cloud blob storage is an injection-point change, not a controller change.

### 3.3 UC3: Automated Reviewer Assignment

**Controller.** `ReviewerAssignmentController` plays the Controller role, but UC3 is distinct from the other two in that no human initiates the flow — the trigger is an internal system event published when a research output is submitted. In GRASP terms this is a system-event controller. It receives a bare output ID and must retrieve everything else it needs before beginning the rule pipeline.

**Creator.** The `Assignment` entity is created by `ReviewerAssignmentController` because the controller is the only class that accumulates all the necessary pieces at the same time — output ID, selected reviewer IDs, timestamp, and the rationale string documenting why those reviewers were chosen. The self-call to `buildAssignment()` formalises this construction once all prerequisites are in place.

**Information Expert.** Information expertise in UC3 is well contained. `ReviewerPool` knows the full set of available reviewers and how to match them to a research domain. `ConflictChecker` holds the three conflict rules — co-authorship, institutional affiliation, and recency of collaboration — and applies them without leaking any rule logic to the controller. `WorkloadBalancer` knows the workload threshold and the selection algorithm. The two repository classes (`ResearchOutputRepository`, `AssignmentRepository`) each own their respective data access concerns.

**Indirection.** Three indirection points shield the UC3 controller from infrastructure details. `NotificationService` decouples it from how reviewer emails are actually sent. `ResearchOutputRepository` means the controller never touches the data store for output retrieval directly. `AssignmentRepository` handles all persistence specifics for assignments. Any of the three could change implementation without the controller being affected.

**Polymorphism.** The conflict-checking logic in `ConflictChecker` is the strongest polymorphism candidate in UC3. Right now, three conflict types are implemented as private methods within a single class — workable, but closed to extension. Applying the Strategy pattern (discussed in Task 3) would extract each rule into its own `ConflictCheckStrategy` implementation, making the rules composable and independently testable without touching the class that applies them.

### 3.4 System-Wide Responsibility Evaluation

The following analysis evaluates each GRASP principle across all three use cases, assessing the implications for coupling, cohesion, and scalability at the system level.

#### *3.4.1 Controller*

**Assignment.** Three use-case controllers are defined: `ProjectController` (UC1), `DatasetController` (UC2), and `ReviewerAssignmentController` (UC3). In the refined architecture (Task 4), these extend a common `BaseController`, which holds shared services and defines the canonical workflow template.

**Justification.** Without a use-case controller, a researcher actor would need to know the internal order of operations for project creation — call the validator, then the repository, then the notification service — which is system-internal knowledge that no actor should possess. The controller absorbs this coordination responsibility and presents a single, simple entry point to the outside world.

**Coupling Evaluation.** Controllers are inherently coupled to their collaborating services

— that is unavoidable. What the design manages is the *nature* of that coupling: all dependencies are injected as interface-typed references rather than concrete instances. The result is low afferent coupling (almost nothing depends on the controller directly) and manageable efferent coupling (the controller relies on abstractions it does not own).

**Cohesion Evaluation.** Each controller is scoped to exactly one use case. `ProjectController` has a single purpose — coordinating the UC1 workflow — and the same holds for the others. Mixing use-case logic into a shared controller would break this cohesion immediately.

**Scalability.** Adding a new use case means creating a new controller that extends `BaseController` and wires its domain-specific services. None of the existing controllers are touched. This is an open/closed boundary in practice, not just in principle.

### *3.4.2 Creator*

**Assignment.** `ProjectController` creates `Project` (UC1); `DatasetController` creates `Dataset` (UC2); `ReviewerAssignmentController` creates `Assignment` (UC3).

**Justification.** In all three use cases the controller ends up holding all the data needed to build the domain entity — either because it received it directly, or because it assembled it from the outputs of preceding service calls. Centralising construction there means entities are only ever built from validated, complete input.

**Scalability.** This is a good starting position that scales naturally. If `Project` construction ever becomes complex enough to warrant a dedicated factory, that factory can be extracted from the controller without any change to the public interface. The controller's responsibility shifts from "build directly" to "delegate to factory", and nothing else in the system changes.

### *3.4.3 Information Expert*

**Assignment.** The Information Expert principle is applied throughout the design: each class is assigned responsibilities that it can fulfil using its own data, without needing to access another class's internal state. Key assignments are summarised in Table 1.

Table 1: Information Expert Assignments

| Class | Information Expert For |
|---|---|
| `ValidationService` | Project validity rules (mandatory fields, date constraints) |
| `FormatValidator` | File format detection and format acceptance rules |
| `IntegrityScanner` | File integrity checking (checksum, corruption detection) |
| `MetadataService` | Metadata extraction from file formats |
| `ReviewerPool` | Reviewer-to-domain expertise matching |
| `ConflictChecker` | Conflict-of-interest rules (co-authorship, institutional, recency) |
| `WorkloadBalancer` | Workload threshold rules and reviewer selection optimisation |
| `ProjectRepository` | Project persistence and retrieval |
| `StorageService` | File blob storage and storage-reference generation |
| `DatasetRepository` | Dataset record persistence and retrieval |
| `AssignmentRepository` | Assignment persistence and retrieval |
| `AuditLogger` | Audit event recording across all three use cases |

**Justification.** The key benefit is avoiding data-extraction chains that make code brittle. Rather than the controller pulling reviewer IDs and author lists to compare them itself, `ConflictChecker.hasConflict()` receives whole objects and works with their data directly. Logic lives where the data lives, and the controller stays clean.

**Scalability.** When rules expand — new conflict types, stricter validation requirements — the growth is contained to the class that owns the relevant data. `ConflictChecker` gets new strategies; `ValidationService` gets new rules; the controllers stay unchanged. This is a direct consequence of applying the Information Expert principle consistently.

#### *3.4.4 Low Coupling*

**System-Wide Assessment.** The design achieves low coupling through three mechanisms: interface-based dependencies, dependency injection, and zero service-to-service coupling. All inter-service communication is mediated by the controller. Each concrete controller has 3–4 unique service dependencies plus 2 inherited from `BaseController`; no service class has more than 0–1 dependencies on other classes.

**Scalability.** This architecture makes individual components replaceable in isolation. Switching from SMTP to a third-party notification API means modifying `NotificationService` only. Moving from a relational store to a document database means rewriting the relevant repository only. Controllers and peer services are unaffected in either case.

#### *3.4.5 High Cohesion*

**System-Wide Assessment.** Every non-controller class performs a single, well-defined function; controllers are scoped to use-case coordination. A naïve monolithic `ResearchService` would mix validation, persistence, role assignment, notification, integrity scanning, conflict checking, and workload balancing — any change to one concern would ripple across the whole class.

**Scalability.** High cohesion bounds growth naturally. New validation rules go into

`ValidationService`; new conflict types go into `ConflictChecker` (or, better, into a new `ConflictCheckStrategy` implementation). Controllers remain stable regardless of how much the service classes evolve beneath them.

#### *3.4.6 Indirection*

**System-Wide Assignment.** `NotificationService` is the primary system-wide indirection layer, decoupling two controllers from delivery mechanisms. `AuditLogger` provides indirection between all three controllers and log storage. All repository classes provide indirection between controllers and the data persistence infrastructure.

**Scalability.** The indirection layer is what makes infrastructure migration low-risk. When the platform eventually needs WebSocket push notifications, or moves dataset storage to a different cloud provider, those changes stop at the indirection boundary. Controllers and domain classes above it are entirely unaffected.

#### *3.4.7 Polymorphism*

**Assignments.** Polymorphism is applied at two levels. First, all validators (`ValidationService`, `FormatValidator`) implement the `Validator<T>` interface, enabling the `BaseController` to reference validators polymorphically. Second, the Strategy pattern applied to `ConflictChecker` (proposed in Task 3) introduces `ConflictCheckStrategy` implementations for each conflict type, enabling rules to be composed and substituted at runtime.

**Scalability.** The alternative to polymorphism is modifying `ConflictChecker` or the validation logic every time a new rule is needed — touching tested, deployed code. With `ConflictCheckStrategy` and `Validator<T>` in place, a new rule is a new class. Existing classes are not touched and therefore cannot regress.

#### *3.4.8 System Modularity Improvement*

Modularity is most usefully measured as the degree to which a system can be decomposed into independently developable, testable, and replaceable units. The GRASP-based design improves system modularity along three dimensions.

**Decomposition.** The design decomposes naturally along two axes: by concern (each class owns one concern), and by use case (each controller owns one workflow). No class crosses these boundaries. This means any component can be understood, modified, or replaced in isolation without reading the rest of the system.

**Interface contracts.** The `Validator<T>`, `Repository<T>`, `FileStorage`, and `ConflictCheckStrategy` interfaces define formal boundaries between modules. A concrete implementation and its replacement share no implementation detail — only the interface contract. Concrete classes behind these boundaries are interchangeable modules.

**Quantified impact.** In the naïve design (one monolithic service per use case), a change to audit logging requires modifying three classes. In the optimised design, it requires modifying one (`AuditLogger`). The same holds for notification delivery, repository technology, and validation rules. The number of classes that must change for any given modification has been reduced from three to one across all cross-cutting concerns — a direct, measurable improvement in modularity.

## 4 Task 3 – Modelling System Behaviour

Each use case selected for this report represents a distinct system behaviour archetype. Identifying these archetypes is essential for selecting the most appropriate modelling strategy and avoiding the use of sequence diagrams for concerns they are ill-suited to represent.

### 4.1 UC1: Event-Driven System

**Classification.** UC1 is best characterised as an event-driven use case. A single researcher action — submitting project details — triggers a cascade of downstream activity: validation, persistence, role assignment, notifications, and audit logging. More importantly, the `Project` entity itself has a lifecycle with distinct states (`DRAFT`, `ACTIVE`, `COMPLETED`, `ARCHIVED`) that are driven by events over time, not just at creation.

**Modelling Challenges.** Sequence diagrams struggle with event-driven behaviour for two reasons. The first is **state explosion**: as more features are added to project management, the number of possible states and valid transitions multiplies. A sequence diagram can only show one path, so it is structurally incapable of representing the full state space. The second is concurrency: what happens when a collaborator accepts an invitation while another is simultaneously removed? The sequence diagram has no natural representation for this, yet it is exactly the kind of race condition that matters in a real system.

**Proposed Modelling Strategy.** A **UML State Machine Diagram** is the right complementary tool here. It models the `Project` entity's full lifecycle: what states it can be in, what events drive transitions between them, and what guard conditions prevent invalid transitions. This is the type of information that sequence diagrams are fundamentally not designed to capture.

**Model Adaptation.** The state machine diagram (Figure 5) makes immediately visible how much the UC1 sequence diagram leaves out. The sequence shows only the creation path (`DRAFT` → `VALIDATING` → `ACTIVE`). The `COMPLETED` and `ARCHIVED` states, and all the events that drive transitions into them, simply do not exist in the sequence diagram's view of the system.

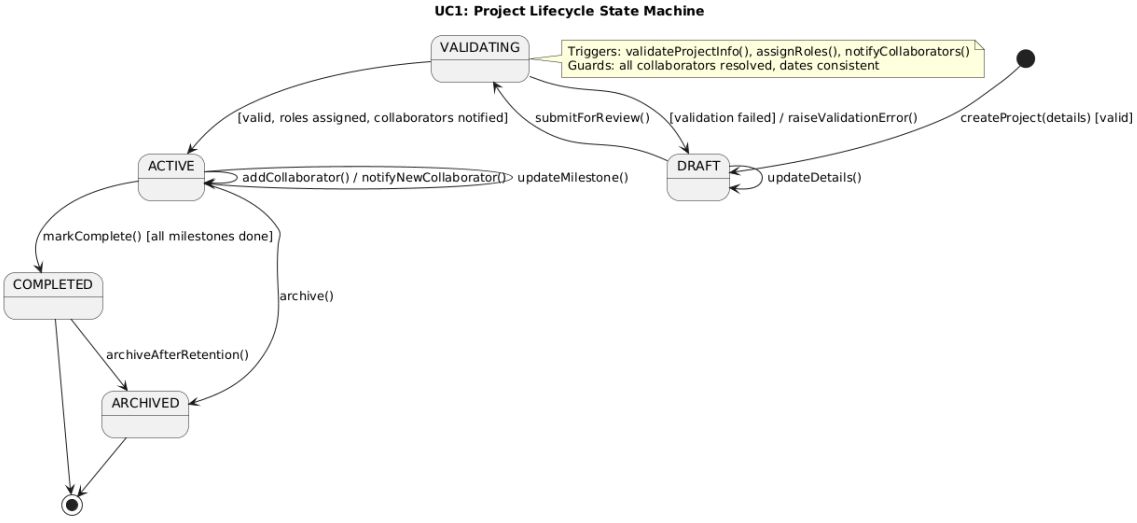


Figure 5: UC1: Project Lifecycle State Machine

### 4.2 UC2: Data-Processing Pipeline

**Classification.** UC2 is a textbook data-processing pipeline. A raw file goes in and a stored, indexed, linked `Dataset` entity comes out — and between those two points the data passes through format validation, integrity scanning, metadata extraction, entity construction, blob storage, and relational persistence. The output is categorically different from the input; the system's job is to transform and enrich the data at each stage.

**Modelling Challenges.** Three issues arise when modelling UC2 purely as a sequence diagram. Deep nesting of `alt` fragments — one inside another — makes the diagram difficult to read once the pipeline exceeds three or four stages. Stage extensibility is also a problem: inserting a new step (a virus scan, for example) between existing stages means modifying the controller, which should not change when a new pipeline stage is added. Finally, the sequence diagram implies that all stages run in strict order, hiding the fact that format validation and integrity scanning could in principle run in parallel.

**Proposed Modelling Strategy.** The **Chain of Responsibility pattern** addresses the extensibility problem by representing each pipeline stage as a handler that either processes the request and passes it on, or short-circuits and returns a failure. Adding a stage means inserting a new handler — the controller does not change. The **UML Activity Diagram**, especially with swimlanes, directly addresses the readability problem: decision diamonds express error-exit paths more cleanly than nested `alt` blocks, and fork/join nodes can make any parallel stages explicit.

**Model Adaptation.** As the activity diagram (Figure 6) shows, swimlanes make component responsibility immediately visible at a glance. Decision diamonds handle error exits without nesting, and the overall flow reads top-to-bottom rather than requiring the reader to trace through nested `alt` blocks. The structural advantages over the sequence diagram are significant for a use case of this complexity.

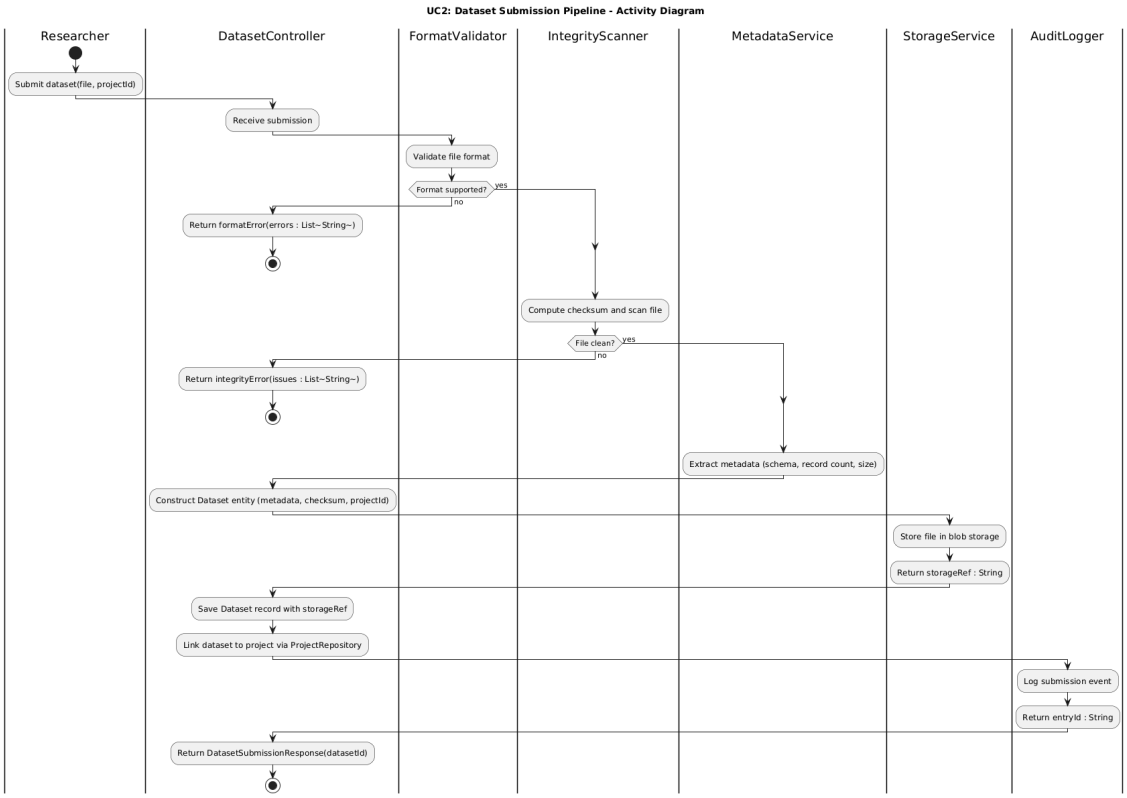


Figure 6: UC2: Dataset Submission Pipeline – Activity Diagram

### 4.3 UC3: Rule-Based Decision System

**Classification.** UC3 stands apart from the other two use cases: its output is not the result of a data transformation, but of a set of business rules applied to a set of candidates. Domain matching, conflict-of-interest filtering, and workload balancing each eliminate or select reviewers. The final assignment is whatever candidates survive all three rule sets.

**Modelling Challenges.** Rule-based systems have one persistent maintenance problem: rules accumulate. The conflict-of-interest rules in `ConflictChecker` currently number three, but research ethics requirements evolve and new rules are added over time. Embedding them all as private methods in one class produces a growing method that is hard to read and impossible to test in isolation. The sequence diagram compounds this by implying the rules execute in a fixed, sequential order, which is misleading if the system ever moves to a rules-engine approach where execution order is not guaranteed.

**Proposed Modelling Strategy.** The **Strategy pattern** is a natural fit here. Each conflict rule becomes its own class implementing a `ConflictCheckStrategy` interface, and `ConflictChecker` becomes a compositor that iterates and applies whatever strategies are registered. Adding a new rule means writing a new class and registering it — the compositor and all existing rules are untouched.

**Model Adaptation.** The class diagram (Figure 7) shows the result: `CoAuthorshipConflict`, `InstitutionalConflict`, and `RecentCollaborationConflict` each implement `ConflictCheckStrategy` independently. `ConflictChecker.filterConflicts()` iterates the registered strategies and removes any reviewer for whom any strategy returns true. Each rule is independently testable, and the open/closed boundary is formally drawn — future rules extend the

system rather than modifying it.

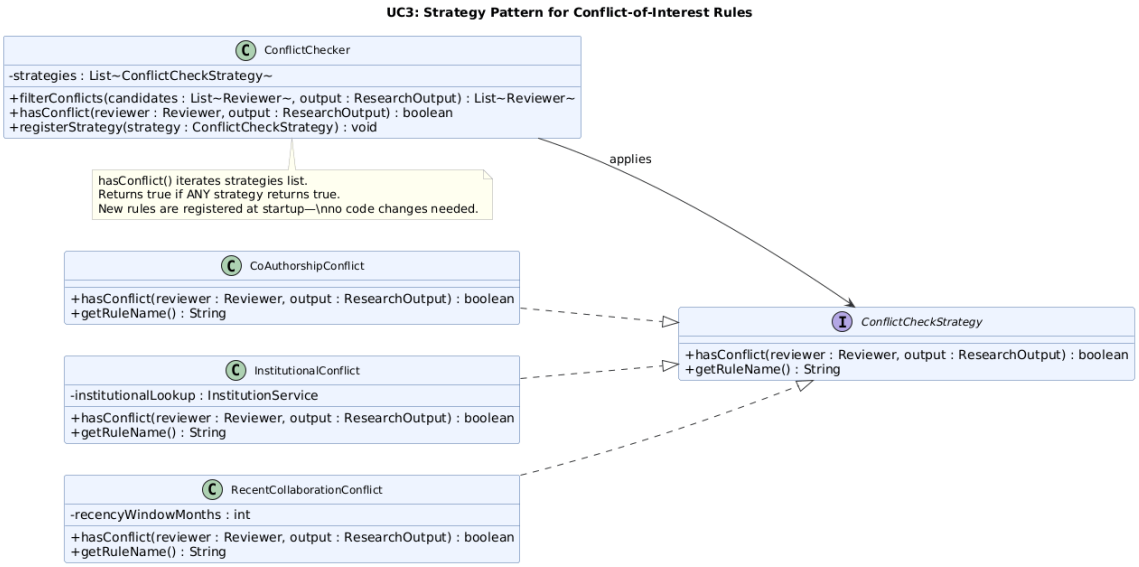


Figure 7: UC3: Strategy Pattern for Conflict-of-Interest Rules

# 5 Task 4 – Design Optimisation

This task draws together the threads from the first three tasks to make a case for a more generalised architecture. The argument is cumulative: the redundancies in Task 1 suggest shared abstractions, the GRASP analysis in Task 2 justifies why those abstractions are principled, and the behaviour classification in Task 3 reveals further modelling opportunities. The optimisation addressed here spans model reuse, shared interaction patterns, and the structural simplification that follows from a `BaseController` hierarchy.

## 5.1 Model Reuse Strategies

**Generic Validator Interface.** A `Validator<T>` interface with a single method — `validate(entity: T): ValidationResult` — delivers several concrete reuse benefits:

- `BaseController` can declare a `Validator<T>` field, giving all concrete controllers a consistent, inherited mechanism for input validation.
- The validation interaction fragment — delegate to validator, check result, terminate on failure — can be written once in a canonical interaction overview diagram and referenced via `«ref»` in each use-case sequence diagram.
- `ValidationService` and `FormatValidator` implement the interface cleanly. `IntegrityScanner` does not — its `scan()` method returns a `ScanResult`, not a `ValidationResult` — so it is wired directly to `DatasetController` as a secondary check outside the generic validation contract.
- New validators for future use cases are introduced by implementing the interface — no changes to the controller framework are needed.

**Generic Repository Interface.** A `Repository<T>` interface with methods `save(entity: T): T`, `findById(id: String): Optional<T>`, and `update(entity: T): T` unifies all repository interactions. `ProjectRepository` and `AssignmentRepository` follow this contract di-

rectly. `DatasetRepository` is a domain-specific elaboration: its `save()` carries an additional `storageRef: String` parameter, atomically binding the cloud storage URI at insert time.

**Shared Services as System-Level Singletons.** `NotificationService` and `AuditLogger` are used across multiple use cases and contain no use-case-specific logic. Designing them as shared system-level services — injected into `BaseController` rather than each concrete controller — eliminates three instances of these dependencies and ensures all controllers share the same notification and audit infrastructure.

### 5.2 Shared Interaction Patterns

The four interaction patterns identified as redundant in Task 1 can be formalised as canonical descriptions and referenced in use-case-specific diagrams using UML's «ref» interaction reference mechanism. This eliminates the repetition at the diagram level, not just the code level.

**Canonical Pattern 1: Validation Gate**

- Controller calls `Validator<T>.validate(request)`
- Controller checks `ValidationResult.isSuccess()`
- On failure: controller throws a domain-specific exception; flow terminates
- On success: flow continues to the next step

This three-step pattern appears in UC1 (once), UC2 (twice — format and integrity), and would appear in all future use cases.

**Canonical Pattern 2: Repository Persist**

- Controller calls `Repository<T>.save(entity)`
- Repository returns the persisted entity with an assigned identifier
- Controller uses the returned identifier in subsequent steps

Note: UC2 extends this pattern — after `StorageService` returns a `storageRef`, `DatasetController` passes both the assembled `Dataset` entity and the `storageRef` to `DatasetRepository.save(dataset, storageRef)`, binding the storage URI atomically at insert time.

**Canonical Pattern 3: Notification Dispatch**

- Controller calls `NotificationService.notify(targets, event)`
- `NotificationService` delivers to all targets and returns `status: NotificationStatus`
- Controller continues to the next step (audit)

Note: this pattern applies to UC1 (`notifyCollaborators`) and UC3 (`notifyReviewers`) only. UC2 has no notification step.

**Canonical Pattern 4: Audit Logging**

- Controller calls `AuditLogger.logEvent(eventType, entity)`
- `AuditLogger` records the log entry and returns `entryId: String`
- Controller returns the response to the caller

## 5.3 Generalised Controllers and Services

**The BaseController.** The structural centrepiece of the optimisation is an abstract `BaseController` that defines the shared workflow skeleton: validate, persist, notify, audit. Concrete controllers — `ProjectController`, `DatasetController`, `ReviewerAssignmentController` — extend it and add their domain-specific steps in the appropriate places. This is the **Template Method pattern**: the invariant parts of the algorithm live in the base class, and subclasses fill in only what varies.

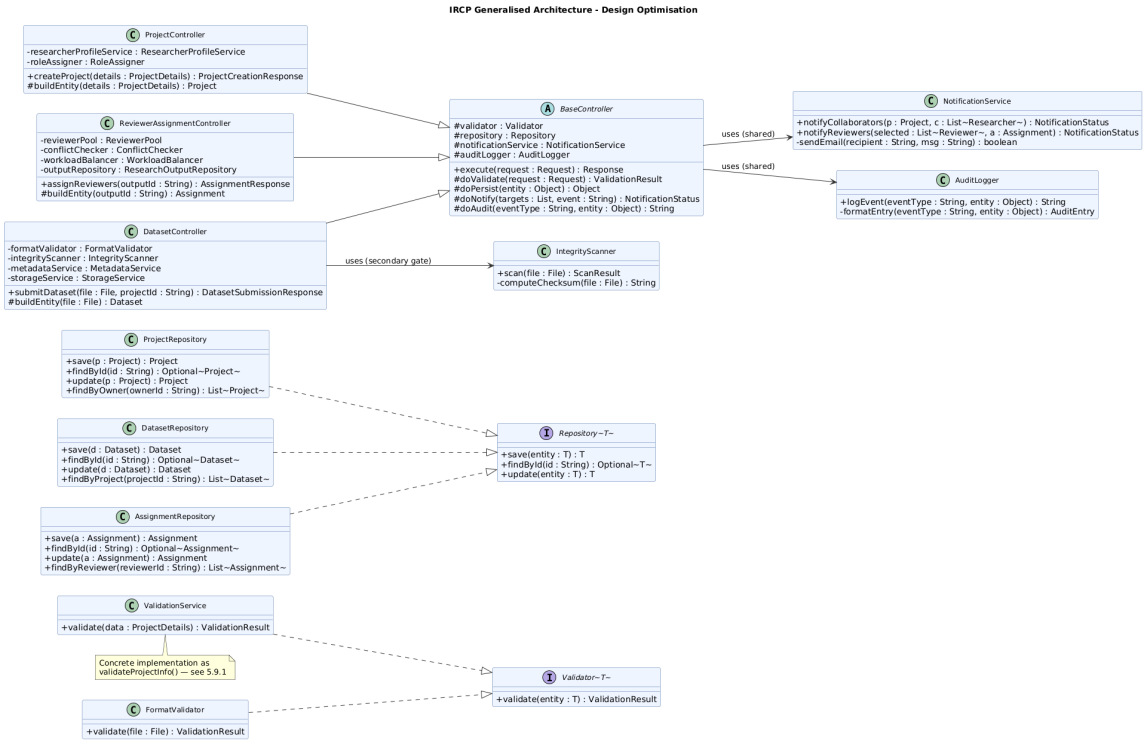


Figure 8: IRCP Generalised Architecture – Design Optimisation

## 5.4 Reduction of Modelling Complexity

**Quantitative Reduction.** Across three use cases, the sequence diagrams collectively contain around 39 message pairs before generalisation — roughly 13 per use case. Introducing `«ref»` fragments for the four canonical patterns brings this to approximately 27 (a 30% reduction), and expressive coverage actually increases: the shared audit logging pattern, previously absent from UC2 and UC3, is now consistently represented across all three use cases. The diagrams become shorter *and* more complete simultaneously.

**Cognitive Complexity Reduction.** A developer joining the team to implement a new use case (e.g., UC4: Collaboration Monitoring) no longer needs to design a controller, validator, repository, notification flow, and audit flow from scratch. They inherit these from `BaseController` and implement only the domain-specific logic. This reduces the time required to implement a new use case by eliminating the re-design of approximately 50% of the controller's interaction structure.

**Diagram Complexity Reduction.** The class diagram for the generalised architecture (Section 5.3) is the single source of truth for the system's structural design. Individual use-case class diagrams (Sections 5.9.1, 5.9.2, 5.9.3) detail only the domain-specific classes

for each use case.

### 5.5 How Improved Modelling Reduces Development Effort

The development effort savings operate at three levels.

**Inheritance of shared infrastructure.** A developer implementing a new use case does not design a validation flow, a persistence interaction, a notification mechanism, and an audit trail from scratch — those are inherited from `BaseController`. If a typical use case has four shared steps and three unique ones, the controller layer alone inherits its way past roughly half the implementation work.

**Reduced test surface.** Without the base class, each controller requires tests for validation handling, persistence invocation, notification dispatch, and audit logging — four test concerns per controller. With `BaseController`, those four concerns are tested once in `BaseControllerTest`. Across three use cases, eight redundant test categories are eliminated.

**Standardised patterns accelerate onboarding.** When every controller follows the same validate–persist–notify–audit skeleton, a developer new to the codebase can orient themselves in any use case's implementation after understanding the base class once. Idiosyncratic, per-controller designs require full re-reading; a standardised hierarchy does not.

**Per-use-case design contrast.** In UC1, the alternative of bundling validation, persistence, role assignment, notification, and audit into a single `ProjectService` would produce a God Object where every concern is entangled with every other. In UC2, placing format validation before the integrity scan reflects a simple performance observation: reject cheap before expensive. A file with the wrong format always fails, and catching it before the checksum computation saves I/O on every such rejection.

### 5.6 How Improved Modelling Supports Maintainability

**Single Change Point for Cross-Cutting Concerns.** Updating the audit entry format means editing `AuditLogger` once; the change propagates automatically to every use case through inheritance. In a design with three independent controllers, the same update would need to be applied and tested three times, with the risk that they gradually drift apart.

**Isolated Change Domains.** Because each class has a single, well-defined concern, the scope of any change is predictable. A change to the conflict-of-interest rules for reviewer assignment affects only `ConflictChecker` (and potentially its `ConflictCheckStrategy` implementations). It does not affect `WorkloadBalancer`, `ReviewerPool`, `NotificationService`, or any other class.

**Regression Risk Reduction.** The combination of high cohesion, low coupling, and isolated change domains directly reduces regression risk. When `ConflictChecker` is modified, unit tests for `WorkloadBalancer`, `ProjectController`, and `DatasetController` do not need to change or be re-run (beyond a standard regression suite).

**Message ordering as a maintainability contract.** The UC1 message ordering is not incidental. Validation must come first: if the input is invalid, no side effects should occur. Persistence must precede notification: sending emails about a project that has not yet been committed to the database would be a reliability failure. Audit logging comes last so the recorded state reflects the fully committed outcome. Encoding this ordering in the

sequence diagram makes it explicit and enforceable.

### 5.7 How Improved Modelling Improves System Evolution

**New Use Cases.** The platform specification includes five use cases, of which this report addressed three. Adding UC4 (Collaboration Monitoring) and UC5 (Research Output Evaluation) requires only: (1) creating a new controller extending `BaseController`; (2) creating domain-specific services; and (3) implementing use-case-specific validators and repositories by extending the existing interfaces.

**Future: AI-Assisted Reviewer Matching.** The proposed AI extension to UC3 — replacing or augmenting `ReviewerPool.findCandidates()` with a semantic similarity model — is accommodated without architectural change. `ReviewerPool` is an injected dependency; an `AIReviewerPool` class implementing the same interface can be injected at startup.

**Future: Real-Time Collaboration Events.** If the platform evolves to support real-time notifications (WebSocket push) alongside email, `NotificationService` can be extended with a new delivery strategy without modifying any controller.

**Future: Ethics Compliance Integration.** A new `EthicsComplianceValidator` implementing `Validator<ProjectDetails>` could be injected alongside `ValidationService` in `ProjectController` to enforce ethics-board requirements.

**Anticipatory design across use cases.** Two details in the current design illustrate how explicit modelling surfaces anticipatory design opportunities. The checksum computed by `IntegrityScanner` in UC2 is embedded in the persisted `Dataset` entity even though nothing in the current workflow reads it back — when a download-and-verify feature is eventually added, the infrastructure is already in place. Similarly, the `rationale` field on `Assignment` in UC3 documents which conflict rules fired and why a particular reviewer was chosen. No current workflow reads it, but it will be immediately useful for audit queries, dispute resolution, and research ethics reviews. Both are design decisions that only become visible when interactions are modelled explicitly rather than collapsed into monolithic service calls.

### 5.8 Design Trade-Offs and Critique

**Trade-Off 1: Abstraction vs. Comprehensibility.** The layers of abstraction introduced here — `BaseController`, `Validator<T>`, `Repository<T>`, `ConflictCheckStrategy` — carry a real onboarding cost. A new developer cannot write a use case without first understanding the inheritance hierarchy and the generic interfaces. For a small team building a prototype, this overhead may genuinely slow early sprints. The argument for accepting it is that the platform is intended for long-term, production-scale use with an evolving rule set, and the abstraction cost is front-loaded while the maintenance savings compound over time.

**Trade-Off 2: Controller as Bottleneck.** The hub-and-spoke structure — everything routed through the controller — simplifies the interaction model, but the controller becomes a potential bottleneck under high concurrency. The mitigation is straightforward: keep controllers stateless. No mutable instance state between requests means multiple controller instances can run concurrently without coordination, which is how the current design is structured.

**Trade-Off 3: WorkloadBalancer Snapshot Staleness.** The `Reviewer.currentWorkload` field is an in-memory snapshot that can become stale under concurrent assignment. Two

simultaneous assignment runs could both read the same workload value before either commits its result, causing the same reviewer to be over-assigned. The clean solution is to have `WorkloadBalancer` query live workload from `AssignmentRepository` at selection time rather than relying on a cached field. This adds a database read per assignment but eliminates the race condition. Given the academic integrity implications, the design should prioritise correctness over latency.

**Critique.** Two gaps in the current models are worth being transparent about. The first is **authentication and authorisation**: none of the sequence diagrams include an `AuthorisationService` check before the controller begins its workflow. In a production system, verifying that the calling researcher has permission to create a project or submit a dataset is non-negotiable, and it belongs in `BaseController` as the very first operation of `execute()`, before even validation runs.

The second gap is **transaction boundaries**. The UC1 sequence shows persistence and notifications as separate steps — but if `NotificationService.notifyCollaborators()` fails after `ProjectRepository.save()` has already committed, the system is left with a persisted project whose collaborators know nothing about it. An outbox pattern — where notification events are written transactionally alongside the entity, then dispatched asynchronously — would solve this without holding a database transaction open during network calls. Both gaps are acknowledged limitations of the current model scope.

## 5.9 Domain Class Diagrams per Use Case

The three class diagrams in this section detail the structural design for each use case individually. They are positioned here in Task 4 because they are most useful when read alongside the generalised architecture diagram (Section 5.3): that diagram shows the shared abstractions, while these show how each use case's specific classes fit into the broader hierarchy. Together they give a complete structural picture of the system.

### *5.9.1 UC1: Create Research Project – Class Diagram*

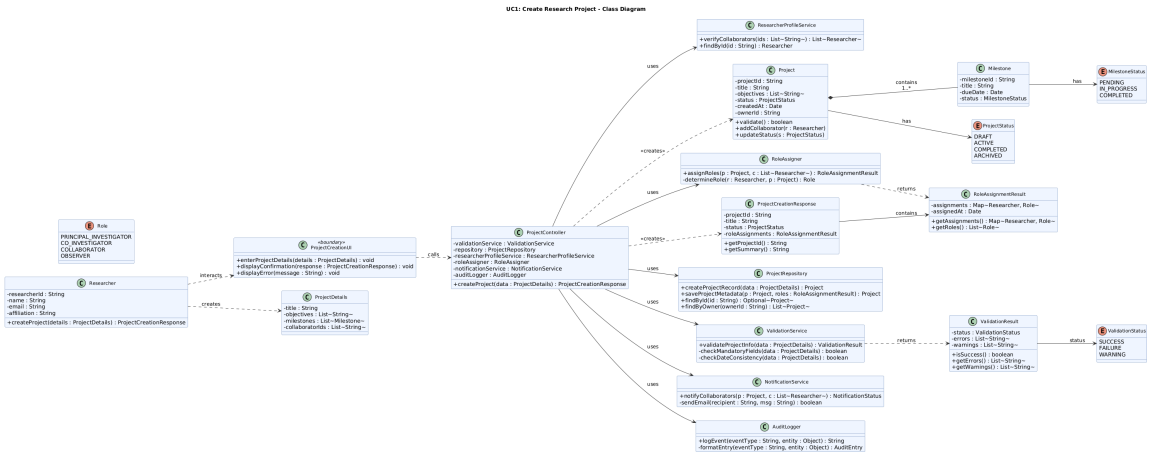


Figure 9: UC1: Create Research Project – Class Diagram

### 5.9.2 UC2: Submit Dataset – Class Diagram

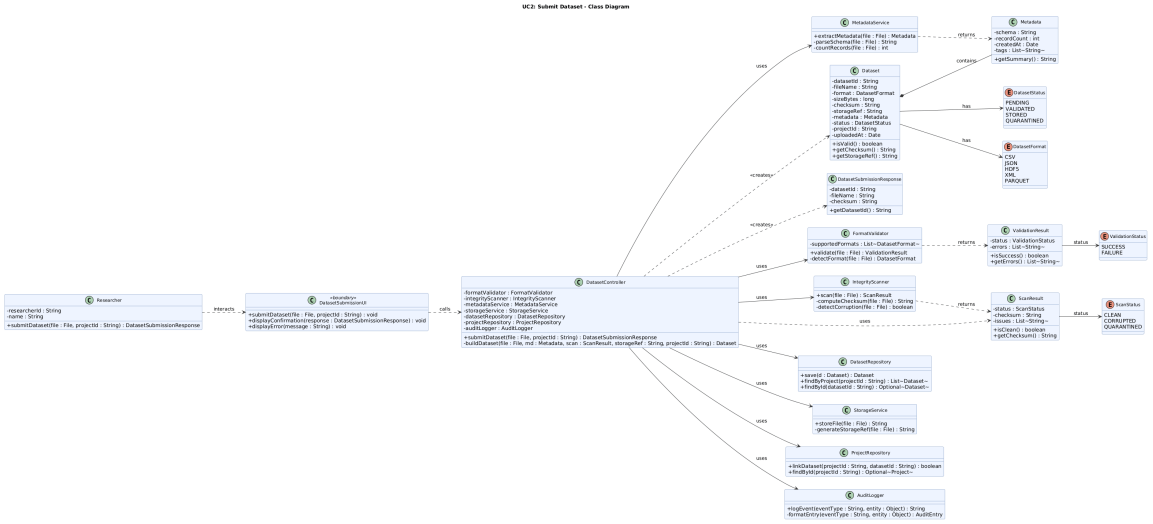


Figure 10: UC2: Submit Dataset – Class Diagram

### 5.9.3 UC3: Automated Reviewer Assignment – Class Diagram

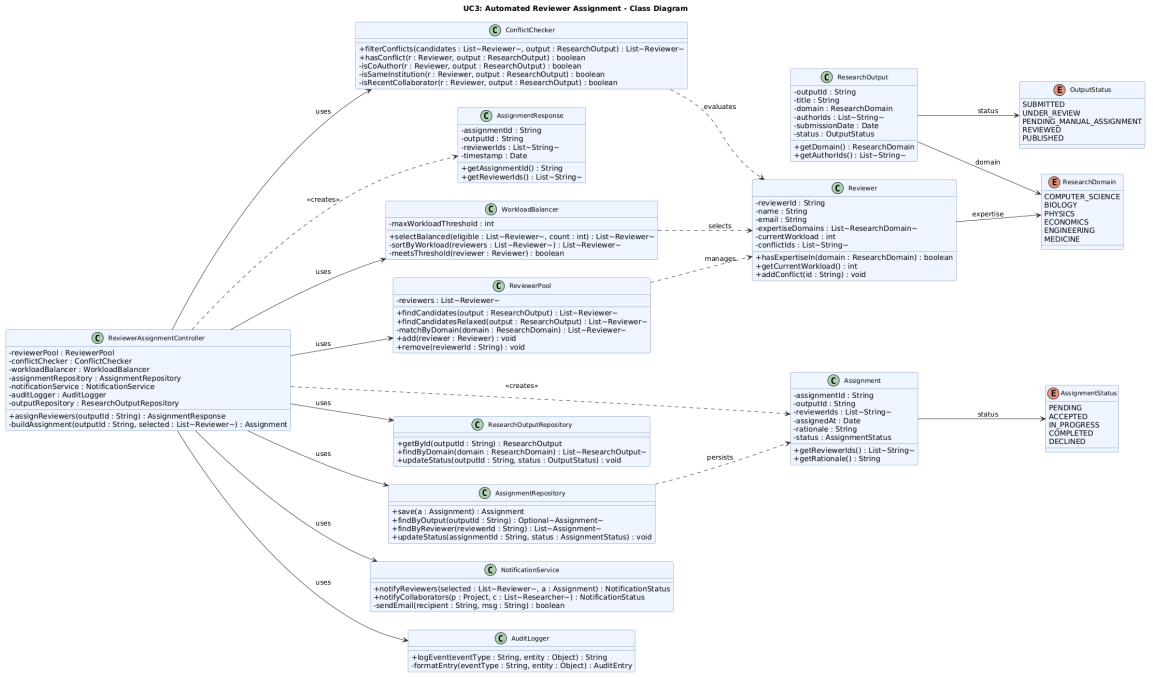


Figure 11: UC3: Automated Reviewer Assignment – Class Diagram

## 6 Conclusion

Across the four tasks, this report has modelled and analysed three use cases that differ in their behaviour type, their complexity, and the modelling techniques best suited to them. The sequence diagrams provide a precise account of message flows and decision points; the class diagrams specify the structural design that supports them; and the complementary diagrams in Task 3 — the state machine, activity diagram, and strategy class diagram — address the behavioural dimensions that sequence diagrams cannot capture.

The GRASP analysis in Task 2 grounds each design decision in an established responsibility-assignment principle rather than intuition. Low coupling, high cohesion, and the consistent application of Information Expert, Indirection, and Polymorphism produce a system where components have clear, bounded roles: nothing knows more than it needs to, and nothing is responsible for more than one concern.

The optimisation in Task 4 follows naturally from the redundancies in Task 1 and the GRASP analysis in Task 2. The `BaseController` template, the generic `Validator<T>` and `Repository<T>` interfaces, and the shared `NotificationService` and `AuditLogger` turn what would otherwise be three separate, partially overlapping controller designs into a coherent hierarchy with a clear extension point for future use cases.

The trade-offs and critique in Section 5.8 are deliberate. Abstraction levels carry onboarding costs; the hub-and-spoke pattern creates potential bottlenecks; workload snapshot staleness is a real correctness risk; and the current models omit authorisation and transaction boundaries. These are acknowledged limitations, and each points toward a concrete architectural improvement for the next iteration of the system.